Case Study 2.28: Secure Function Calling in Large Language Models

A Complete Solution Design covering Architecture, Training, Prompting and Security

1. Introduction and Problem Context

Modern enterprise applications increasingly connect Large Language Models (LLMs) to external tools, APIs, and databases through "function calling," allowing a model to translate a natural-language request into a structured, executable action (e.g., booking a ticket, querying a database, or transferring funds). While this capability greatly expands what an LLM can do, it also converts the model from a passive text generator into an active agent that can trigger real-world side effects. The case study therefore requires a solution that is functionally correct and also secure against prompt injection, privilege abuse, malformed calls, and other adversarial or accidental edge cases.

2. LLM Architecture Relevant to the Case

At its core, an LLM used for function calling is a decoder-only Transformer. Input text is tokenized and embedded, positional information is added, and the sequence passes through N stacked Transformer blocks, each combining multi-head self-attention (to model relationships between tokens) with a position-wise feed-forward network, residual connections, and layer normalisation. The final hidden states are projected through a linear layer and softmax to produce a probability distribution over the vocabulary, from which the model can generate either natural language or a structured function-call payload (typically JSON) that a host application executes.

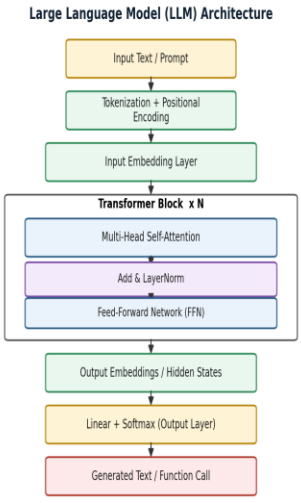

Figure 1: Transformer-based LLM architecture producing text or structured function calls.

3. Pre-training, Fine-tuning and Transfer Learning

3.1 Pre-training

The base model is pre-trained on large, diverse text corpora using a self-supervised objective such as next-token prediction. This stage gives the model broad linguistic and world knowledge but no awareness of a specific organisation's tools or safety policies.

3.2 Fine-tuning

Supervised fine-tuning (SFT) and instruction-tuning are then applied on curated examples of correct tool-use dialogues, teaching the model when to call a function, how to format arguments, and when to refuse. Reinforcement Learning from Human Feedback (RLHF) or similar preference-optimisation methods further align the model's behaviour with safe, helpful responses.

3.3 Transfer Learning

Rather than training from scratch, transfer learning reuses the pre-trained model's general representations and adapts them to the function-calling task with comparatively little task-specific data. This is more cost-effective and preserves the model's general reasoning ability while specialising its tool-use behaviour.

4. LoRA and Parameter-Efficient Fine-Tuning (PEFT)

Full fine-tuning of a multi-billion-parameter LLM is expensive and risks catastrophic forgetting. Low-Rank Adaptation (LoRA), a PEFT technique, freezes the original weight matrix W and injects two small trainable low-rank matrices, A and B, such that the effective update is $W' = W + BA$. Only A and B are trained, reducing trainable parameters by orders of magnitude while keeping the base model intact. This makes LoRA well suited to teaching the model organisation-specific function schemas and secure-calling conventions cheaply and safely, and multiple LoRA adapters can be swapped in for different tool sets without retraining the whole model.

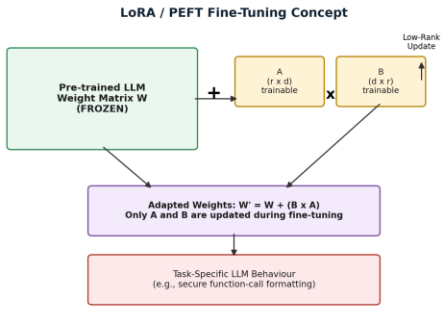

Figure 2: LoRA fine-tuning — base weights frozen, only low-rank matrices A and B are trained.

5. Prompt Engineering Strategy

A layered prompting strategy is used to make function selection reliable and safe.

5.1 Zero-shot, One-shot and Few-shot Prompting

- Zero-shot: the model is given only the function schemas and task instruction; used for simple, well-covered tools.
- One-shot: a single worked example of correct tool invocation is provided to anchor output format.
- Few-shot: multiple examples, including at least one refusal / edge-case example, are provided to calibrate tone, format and safety boundaries for complex or ambiguous tools.

5.2 Reasoning-Oriented Prompting

Chain-of-thought style prompting asks the model to reason step-by-step about whether a function call is necessary, which function fits, and whether the extracted arguments are complete and safe, before emitting the final structured call. This reduces hallucinated or premature function calls.

5.3 Structured Prompting

Function definitions are supplied to the model as explicit JSON Schemas (name, description, parameter types, required fields). The model is instructed to respond only with a JSON object conforming to this schema, which is far more reliably parsed and validated than free-form text.

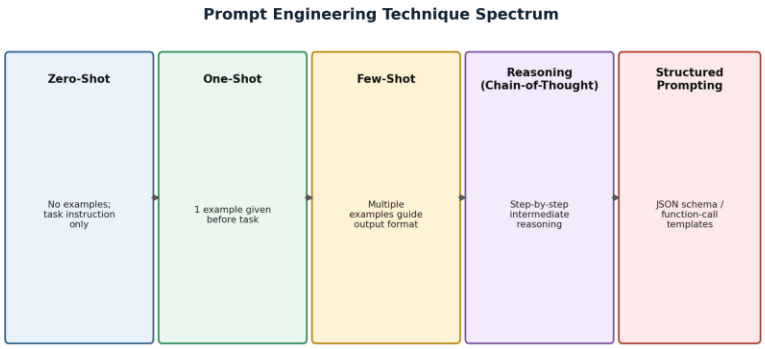

Figure 3: Spectrum of prompting techniques applied to the function-calling pipeline.

6. Secure Function-Calling Solution Design

The proposed architecture places a Security Gateway between the LLM's function-call output and the actual execution environment, so that no LLM-issued call is trusted implicitly.

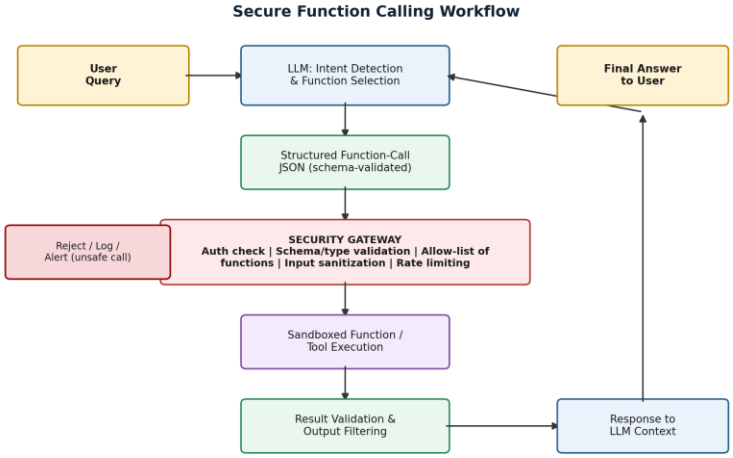

Figure 4: End-to-end secure function-calling workflow with validation, sandboxing and logging.

- Intent detection: the LLM proposes a function name and arguments using structured prompting.
- Schema validation: the gateway checks the JSON against the registered function schema and rejects malformed calls.
- Authorisation: role-based access control confirms the requesting user/session may invoke that function.
- Allow-listing: only pre-registered, vetted functions can be executed; unknown names are auto-rejected.

- Sandboxed execution: the function runs in an isolated environment with least-privilege credentials and timeouts.
- Output filtering: results are checked/redacted before being fed back into the LLM context or shown to the user.
- Logging and monitoring: every call, decision and rejection is logged for audit and anomaly detection.

7. LLM API Selection

API/model selection should weigh native function-calling support, JSON-mode reliability, context window, latency/cost, and data-residency or compliance needs. Models with first-class structured-output and tool-use APIs (rather than prompt-only emulation) reduce parsing errors and are preferred for production secure function calling; smaller, fine-tuned or LoRA-adapted open models may be selected for latency-sensitive or on-premises deployments where data cannot leave the organisation's environment.

8. Prompt Evaluation

Before deployment, prompts and the resulting function calls are evaluated against: functional correctness (right function, right arguments), schema-validity rate, refusal accuracy on out-of-scope or unsafe requests, robustness to adversarial/prompt-injection test cases, and latency/cost. A held-out test suite of benign, ambiguous and adversarial prompts is scored automatically (schema/unit tests) and periodically reviewed by humans, feeding back into few-shot examples or LoRA fine-tuning data.

9. Security and Edge-Case Handling

The table below summarises the principal risks specific to function calling and the corresponding mitigations built into the design.

Security Risk	Description	Mitigation
Prompt Injection	Malicious text in user input tricks the LLM into invoking unintended or unsafe functions.	Input sanitisation, instruction-hierarchy, allow-listing of callable functions.
Schema Violation	LLM returns malformed or type-mismatched JSON arguments for a function call.	Strict JSON-schema validation before execution; reject non-conforming calls.

Security Risk	Description	Mitigation
Privilege Escalation	Function call attempts to access data/resources beyond the user's permission level.	Role-based access control (RBAC) enforced at the gateway, not by the LLM.
Data Exfiltration	Sensitive data returned by a function leaks into the LLM context or final answer.	Output filtering / redaction layer and least-privilege function design.
Hallucinated Function/Args	Model invents a non-existent function name or fabricates parameter values.	Allow-list matching, default-deny routing, human-in-the-loop for high-risk calls.
Denial of Service / Loops	Repeated or recursive function calls exhaust resources or cost budget.	Rate limiting, call-depth limits, timeouts, monitoring and alerting.

10. Justification of the Proposed Solution

The design combines cost-efficient adaptation (LoRA/PEFT) with disciplined prompting (structured, few-shot, reasoning-oriented) and, critically, treats the LLM's output as untrusted input to a conventional security gateway. This mirrors defence-in-depth practice: even if the model is manipulated via prompt injection or hallucinates an unsafe call, schema validation, allow-listing, RBAC and sandboxing prevent unauthorised or destructive execution. The approach is scalable — new tools are added by registering a schema and updating the allow-list rather than retraining the model — and auditable, since every decision point produces a log entry.

11. Conclusion

Secure function calling requires more than a capable LLM; it requires an architecture where model reasoning (Transformer-based generation, fine-tuned via transfer learning and LoRA, guided by structured and reasoning-oriented prompts) is paired with a non-bypassable security layer that validates, authorises and sandboxes every action before it touches real systems. This layered solution satisfies both the functional goal of natural-language tool use and the non-functional requirement of safety at production scale.